# Метод координации одномерных задач: итог работы и проверка

Nov 10, 2026

## Кратко

Метод работает правильно и даёт верное решение. Но вся его скорость идёт от предобусловливателя ADI, а не от самой координации. С тем же ADI обычный GMRES на исходной двумерной задаче делает столько же итераций и работает в 1,3–2,3 раза быстрее.

- **Все три пункта задания выполнены.** Лаплас решается через одномерные задачи с координатором, есть параллельный код для GPU, проведено сравнение на сетке 100 × 100 и на сетках до 1600 × 1600.
- **Метод верен.** Решение совпадает с точным решением разностной схемы до $10^{-11}$.
- **Метод быстр, но благодаря ADI.** Без предобусловливателя координация сходится так же медленно, как обычный CG: 291 итерация против 310 на сетке 100 × 100. С ADI нужно 6–10 итераций на любой сетке.
- **Контрольный опыт.** Тот же ADI, применённый прямо к исходной задаче, без координатора, даёт то же число итераций. Это следует и из алгебры (раздел 4).
- **Новизна.** В нынешнем виде преимущества в скорости нет. Перспективы — в задачах, где исходного двумерного оператора нет как единого целого (раздел 6).

## 1. Задание и что сделано

Задание состояло из трёх пунктов, все три выполнены.

| Пункт задания | Что сделано |
|---|---|
| 1. Уравнение Лапласа в прямоугольнике заменить одномерными задачами $u_{xx} = q$, $v_{yy} = -q$ с координирующей функцией $q$ | Координатор $q$ ищется минимизацией $J = \frac{1}{2}\|u - v\|^2$. Реализованы градиентный спуск, метод сопряжённых невязок (CR), GMRES и GMRES с предобусловливателем ADI. Решение совпадает с точным решением разностной схемы до $10^{-11}$. |

| Пункт задания | Что сделано |
|---|---|
| 2. Написать код для параллельного решения систем на GPU | Все трёхдиагональные системы вдоль строк или столбцов решаются одновременно. Для GPU написана параллельная циклическая редукция (PCR). Код работает на процессоре, на GPU Apple и на NVIDIA. |
| 3. Оценить эффективность на сетке 100 × 100: традиционное двумерное решение против предложенного метода, схемы одного порядка | Все методы решают одну и ту же пятиточечную схему второго порядка. Измерены время, число итераций, ошибка и память. Дополнительно — сетки до 1600 × 1600 и контрольный опыт с ADI. |

Тестовая задача: $\Delta\varphi = 0$ в единичном квадрате, $\varphi = 1$ на верхней стороне и 0 на остальных. Граничное условие $\sin(\pi x)$ не годится: это одна собственная функция, и на ней любой метод сходится неправдоподобно быстро (8 итераций вместо 291).

## 2. Реализация

Код написан на Python и PyTorch. Один и тот же код работает на процессоре, на GPU Apple (MPS) и на видеокартах NVIDIA (CUDA).

- **Пакетная прогонка.** Разложение считается один раз на всю задачу, а все строки или столбцы сетки решаются одним вызовом. Есть три варианта: классическая прогонка Томаса, треугольные матрицы через BLAS и PCR.
- **Параллельная циклическая редукция (PCR).** Прогонка проходит n уравнений одно за другим, а PCR за $\lceil \log_2 n \rceil$ шагов обрабатывает все уравнения сразу. Коэффициенты считаются заранее, поэтому само решение — это $\lceil \log_2 n \rceil$ поэлементных операций. На GPU PCR в 2,6–4 раза быстрее варианта через BLAS.
- **Итерационные методы для координатора.** Реализованы градиентный спуск с сопряжённой задачей для градиента, CGLS, CR и GMRES. Предобусловливатель ADI строится по Писмену–Рэкфорду: 6 шагов с геометрическими параметрами по границам спектра.
- **Обобщения.** Код поддерживает K подсистем, переменные коэффициенты и перенос ветром (адвекцию).

- **Тесты.** Все проверки запускаются командой `python checks/run_checks.py` и проходят. Среди них: PCR совпадает с прогонкой до $5 \cdot 10^{-16}$, а координация совпадает с прямым двумерным решением.

## 3. Результаты замеров

На сетке 100 × 100 координация с ADI в 46 раз быстрее метода Якоби и в 23 раза быстрее первоначального варианта координации. Но обычный CG без предобусловливания на такой сетке всё ещё быстрее. Замер на Apple M3, время — лучшее из трёх запусков, ошибка — относительно точного решения схемы.

| Метод (сетка 100 × 100) | Устройство | Время, с | Итераций | Ошибка |
|---|---|---|---|---|
| Якоби | процессор | 1,19 | 34 800 | $1 \cdot 10^{-8}$ |
| Холецкий, плотная матрица (2 ГБ памяти) | процессор | 1,58 | — | $3 \cdot 10^{-14}$ |
| CG без предобусловливания | процессор | 0,017 | 310 | $7 \cdot 10^{-11}$ |
| Координация без предобусловливания (CR) | процессор | 0,59 | 291 | $6 \cdot 10^{-11}$ |
| Координация, GMRES + ADI | процессор | 0,026 | 6 | $2 \cdot 10^{-11}$ |
| Координация, GMRES + ADI | GPU Apple, float32 | 0,063 | 3 | $3 \cdot 10^{-6}$ |

На видеокарте NVIDIA в двойной точности на сетке 1600 × 1600 (2,55 млн неизвестных) картина такая:

| Метод (сетка 1600 × 1600) | Устройство | Время, с | Итераций | Ошибка |
|---|---|---|---|---|
| CG без предобусловливания | процессор | 183 | 4 550 | $3 \cdot 10^{-9}$ |
| CG без предобусловливания | GPU NVIDIA | 10,4 | 4 550 | $3 \cdot 10^{-9}$ |
| Координация, GMRES + ADI | процессор | 37,4 | 10 | $7 \cdot 10^{-11}$ |
| Координация, GMRES + ADI + PCR | GPU NVIDIA | 3,05 | 10 | $7 \cdot 10^{-11}$ |

У CG число итераций растёт пропорционально числу узлов вдоль стороны: 310 → 4 550. У варианта с ADI оно почти постоянно: 6 → 10. На GPU Apple есть только одинарная точность, и на больших сетках GMRES там буксует: 51 итерация

вместо 9 на сетке 800 × 800. На NVIDIA в двойной точности этой проблемы нет.

Важно: при таком сравнении метод с предобусловливанием наставлен против метода без него. Честное сравнение — с тем же предобусловливателем на исходной задаче, см. раздел 4.

## 4. Контрольный опыт: ускорение даёт ADI

Тот же предобусловитель ADI (6 шагов, те же параметры τ, та же прогонка) применён к обычной пятиточечной схеме без координатора. Число итераций совпало во всех случаях на процессоре, а обычный подход оказался быстрее.

| Сетка | Координация, GMRES + ADI | Исходная задача, GMRES + ADI |
|---|---|---|
| 100 × 100 | 0,027 с, 6 итераций | 0,012 с, 6 итераций |
| 200 × 200 | 0,079 с, 7 итераций | 0,044 с, 7 итераций |
| 400 × 400 | 0,241 с, 8 итераций | 0,155 с, 8 итераций |
| 800 × 800 | 2,19 с, 9 итераций | 1,68 с, 9 итераций |

Процессор Apple M3, двойная точность, критерий $10^{-10}$. Скрипт `adi_direct_vs_coordination.py`.

**Почему так выходит.** Пусть X и Y — разностные операторы $-\partial^2/\partial x^2$ и $-\partial^2/\partial y^2$ (симметричные, положительные). Одномерные задачи и условие согласования дают линейную систему для q:

$$Xu = f_x - q,\quad Yv = f_y + q,\quad u - v = c - Mq,\quad M = X^{-1} + Y^{-1} = X^{-1}(X+Y)\,Y^{-1}$$

Последнее равенство — ключевое: внутри оператора координации сидит исходный двумерный оператор X + Y. Отсюда два следствия.

1. **Без предобусловливателя.** Для прямоугольника X и Y коммутируют, собственные числа M равны $1/\lambda + 1/\mu$. Число обусловленности M выходит тем же, что у X + Y, порядка $1/h^2$. Поэтому координация без предобусловливания сходится как обычный CG: 291 итерация против 310.
2. **С предобусловливателем.** Естественный предобусловитель — обратный к M, то есть $Y(X + Y)^{-1}X$. В нём $(X + Y)^{-1}$ — это решение исходной задачи, его и приближает ADI (обозначим $A_m$):

$$P = Y A_m X,\qquad MP = X^{-1}\,\big[(X+Y)A_m\big]\,X$$

Предобусловленный оператор координации подобен $(X + Y)A_m$ — исходной задаче с тем же ADI. Когда X и Y коммутируют, оба оператора симметричны, и у GMRES совпадает число итераций. Итерация координации при этом дороже: лишние две прогонки на умножение на M вместо дешёвого умножения на пятиточечную матрицу.

Иначе говоря, все цифры «в 20–60 раз быстрее» из раздела 3 — это эффект ADI, а ему больше 60 лет (Писмен и Рэкфорд, 1955). На самой сложной сетке для GPU Apple в одинарной точности обычный подход ещё и устойчивее: 24 итерации против 51 на сетке 800 × 800.

## 5. Что это значит для новизны

Как способ быстро решать уравнение Лапласа в прямоугольнике метод координации нового преимущества не даёт. Рецензент журнала первым делом спросит о сравнении с GMRES + ADI на исходной задаче, и таблица из раздела 4 отвечает не в пользу координации.

При этом метод не ошибочен. Он математически верен, сходится и даёт точное решение. Просто для этой задачи он оказался переформулировкой известного подхода, а не новым способом ускорения.

Идея разбить задачу на подзадачи и согласовать их через минимизацию расхождения известна под несколькими именами (раздел 6). Прежде чем заявлять новизну, стоит сравнить метод с этими работами. Возможно, что какая-то часть постановки действительно новая, но это требует проверки по литературе, которую мы не делали.

## 6. Как можно улучшить и где метод может быть полезен

Ускорить координацией решение задачи, для которой есть обычный двумерный оператор, скорее всего не удастся. Подобие $M{\cdot}P$ и $(X + Y){\cdot}A_m$ из раздела 4 верно для любых обратимых X и Y, не только для прямоугольника. Сила подхода может быть в другом — в модульности:

1. **Разнородные подсистемы.** Например, сеть труб или рек (одномерные модели), связанная с двумерной областью, или подсистемы на разных сетках. Там единого оператора X + Y в готовом виде нет, и координация — естественный способ их связать.
2. **Готовые чужие решатели («чёрные ящики»).** Координации нужно только уметь решать каждую подзадачу. Это позволяет связать существующие программы, не переписывая их.

3. **Координатор только на границах подобластей.** Сейчас q живёт на всей сетке, и неизвестных столько же, сколько в исходной задаче. Если согласовывать подобласти только на их границах, неизвестных станет гораздо меньше. Но это поле декомпозиции областей, где уже есть сильные методы (FETI, методы Шварца).
4. **Обратные задачи и управление.** Функционал J легко дополнить невязкой с измерениями и ограничениями, так что согласование подсистем и подгонка под данные делаются в одной оптимизации.

В любом из этих направлений первый вопрос тот же: что будет, если решить задачу целиком стандартным методом с тем же предобусловителем. Если так сделать нельзя или неудобно, у координации есть настоящее преимущество.

**Что почитать.** Список составлен по памяти, точные выходные данные нужно проверить.

| Тема | Авторы, примерное время | Чем близко |
|---|---|---|
| Метод переменных направлений (ADI) | Peaceman, Rachford, 1955; Douglas | тот самый предобусловитель |
| Метод дробных шагов, аддитивные схемы, суммарная аппроксимация | Яненко (1960-е); Самарский, Вабищевич | замена многомерной задачи одномерными |
| Декомпозиция и координация в иерархических системах | Месарович, Мако, Такахара (1970); Ласдон (1970) | само понятие координации подсистем |
| Метод виртуального управления | J.-L. Lions, O. Pironneau (конец 1990-х) | подзадачи со служебными «управлениями» и минимизация расхождения — по идее ближе всего |
| Декомпозиция области через оптимизацию | Gunzburger, Heinkenschloss, Lee (около 2000) | стыковка подзадач как задача оптимального управления |
| FETI | Farhat, Roux (1991) | стыковка подобластей через дополнительные неизвестные на границах |

## 7. Приложение: загрязнение воздуха и поиск источников

Помимо задания, метод применён к модели распространения загрязнения с ветром, диффузией и распадом. Он сходится за 6–10 итераций и для переменных коэффициентов.

Решена и обратная задача: по измерениям станций найти неизвестный источник (пожар) и уточнить мощности известных. Используются сопряжённые задачи («следы» станций), взвешенные наименьшие квадраты и 95-процентная область доверия. Чтобы избежать «обратного преступления», данные синтезированы на мелкой сетке, а восстановление делается на крупной. Результаты — медиана по 50 наборам шума:

| Станций | Ошибка места пожара, км | Ошибка мощности пожара |
|---|---|---|
| 4 | 5,79 | 104 % |
| 6 | 2,28 | 61 % |
| 8 | 1,72 | 44 % |
| 12 | 1,04 | 25 % |
| 16 | 0,98 | 14 % |
| 24 | 0,50 | 11 % |

Начиная с 8 станций истинное место попадает в 95-процентную область в 92–98 % случаев, как и должно быть.

Честная оговорка: обратная задача сама по себе от координации не зависит. Следы станций можно посчитать любым решателем.

## 8. Файлы проекта

Всё лежит в папке проекта `cuda-learning`. Для запуска нужны Python и PyTorch, проверки запускаются командой `python checks/run_checks.py`.

| Файл | Что в нём |
|---|---|
| `coordinator_laplace_descent.py` | метод координации: постановка, спуск, CR, GMRES, предобусловитель ADI |
| `batched_tridiagonal.py` | пакетная прогонка и PCR для GPU |
| `laplace_gpu_benchmark.py` | сравнение методов (пункт 3 задания) |

| Файл | Что в нём |
|---|---|
| `adi_direct_vs_coordination.py` | контрольный опыт из раздела 4 |
| `multi_coordination.py` | обобщение на K подсистем |
| `eco_pollution.py`, `eco_inverse.py` | загрязнение воздуха и поиск источников |
| `results/laplace_gpu_benchmark.md` | подробный отчёт о замерах со всеми цифрами |
| `RUN_NVIDIA.md` | как запустить на видеокарте NVIDIA |
| `checks/` | тесты |

Раньше была написана статья о методе. Контрольный опыт сделан уже после неё, поэтому её выводы о скорости следует читать с учётом раздела 4 этого документа.